SIEMENS

Capstone Project: A PID-controlled DDR with ethernet IVSI backplane.

By: Abdelrahman Wisam Elbana

This document is an assignment created for educational purposes only. It is not an official document of, nor affiliated with, Siemens.

March 2026

Table of Contents

Introduction 2

System Design 2

Client 1 (Plant/Environment) 2

Robot Kinematics 2

Environment and Real-world Simulation 3

Client 2 (PID Controller) 4

IVSI Backplane 4

Experiments And Results 5

Setup 5

Experiment 1 5

Test 1 PID Gains 5

Test 1 - Spawn 1 5

Test 1 - Spawn 2 6

Test 2 PID Gains 7

Test 2 - Spawn 1 7

Test 2 - Spawn 2 7

Test 3 PID Gains 9

Test 3 - Spawn 1 9

Test 3 - Spawn 2 9

Experiment 2 10

Experiment 3 11

Experiment 4 12

PD controller 12

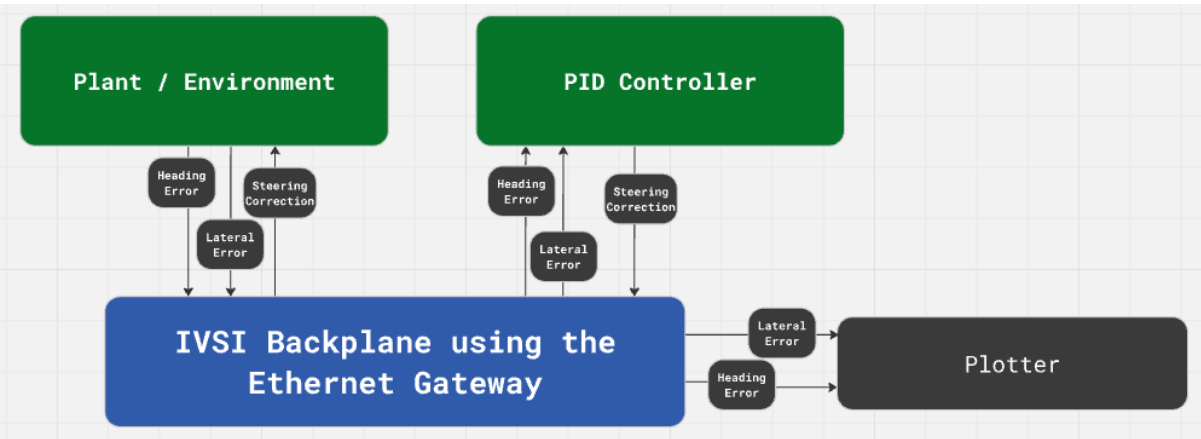
PID controller 12

References 12

Introduction

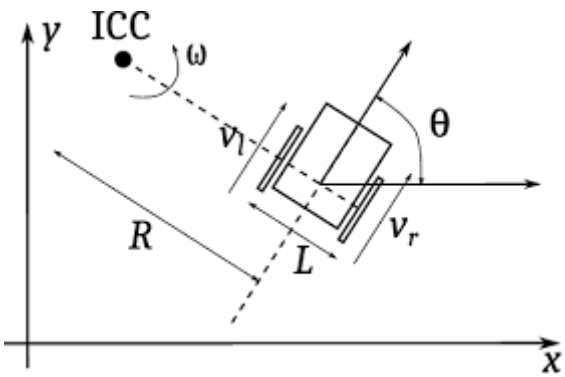
This project presents a line-following differential-drive robot controlled via a distributed three-client architecture communicating over the Innexis Virtual System Interconnect (IVSI) backplane using Ethernet. The primary objective is to steer the robot from an arbitrary spawn location onto a predefined path and track it to completion with minimal lateral error. To ensure a highly realistic simulation environment, the plant explicitly models Differential-Drive Robot (DDR) kinematics, utilizing the Instantaneous Center of Curvature (ICC) to compute the forward kinematics. Furthermore, the simulation actively incorporates real-world imperfections by injecting sensor noise and external disturbances into the system to test robustness. Navigation is managed by a PID controller designed to minimize both cross-track (lateral) and heading errors. By applying a cross-track correction term, the controller seamlessly compiles these positional and rotational deviations into precise velocity commands, enabling stable and accurate trajectory tracking.

System Design



Client 1 (Plant/Environment)

Robot Kinematics



To simulate the robot's movement accurately, the system models the forward kinematics of a differential drive mechanism. We assume the robot is positioned at coordinates (x, y),

heading in a direction that forms an angle θ with the X-axis. The robot's reference frame is centered at a point midway along its wheel axle.

The robot's trajectory is dictated by manipulating the control parameters v_l and v_r, which represent the left and right wheel velocities along the ground, respectively. For the robot to perform a rolling motion, it must rotate about a specific point lying along the common axis of the left and right wheels; this point is known as the Instantaneous Center of Curvature (ICC).

Using the wheel velocities (v_l, v_r) and the distance between the wheel centers L, we first calculate the signed distance from the ICC to the midpoint between the wheels (R) and the rate of rotation about the ICC (ω):

$$R = \frac{l}{2} \cdot \frac{V_l + V_r}{V_r - V_l}, \qquad \omega = \frac{V_r - V_l}{L}$$

Knowing these values, the exact spatial location of the ICC can be found using the following equations:

$$ICC = [x - R\,sin(\theta), y - R\cos(\theta)]$$

To determine the robot's new pose (x', y', θ) at a future time $t + \delta t$, we calculate the motion of the robot as it rotates a distance R about its ICC with an angular velocity of ω. This movement is mathematically modeled by the following matrix equation:

$$\begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = \begin{bmatrix} cos(\omega\delta t) & -sin(\omega\delta t) & 0 \\ sin(\omega\delta t) & cos(\omega\delta t) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x - ICC_x \\ y - ICC_y \\ \theta \end{bmatrix} + \begin{bmatrix} ICC_x \\ ICC_y \\ \omega\delta t \end{bmatrix}$$

Environment and Real-world Simulation

To evaluate the control system under non-ideal conditions, the simulation environment actively models several real-world hardware limitations and physical disturbances. First, to simulate the inaccuracies inherent in physical measuring devices, random noise is injected directly into the system's error calculations, effectively replicating the behavior of faulty or imprecise sensors. The simulation also incorporates a motor time constant (τ) to account for the mechanical inertia of the differential drive system.

In addition to transient noise and delayed actuation, the environment models sustained physical disturbances by introducing a constant wheel slip factor. This simulated slip creates a continuous, biased degradation in the robot's kinematics, manifesting as a persistent ***steady-state error*** that prevents the robot from perfectly tracking the path using proportional or derivative control alone. The introduction of this continuous disturbance specifically validates the necessity of the Integral (I) component within the PID controller. By accumulating the persistent lateral deviation over time, the integral term successfully computes the necessary corrective offsets to counteract the wheel slip, ensuring stable and accurate line-following despite continuous environmental faults.

Client 2 (PID Controller)

The core navigation of the differential-drive robot is governed by a Proportional-Integral-Derivative (PID) controller designed to continuously minimize both lateral and heading deviations. To achieve highly stable trajectory tracking, the control scheme avoids processing positional and rotational errors in isolation. Instead, it formulates a comprehensive error signal by combining the robot's heading error with a Stanley-like cross-track correction term.

The heading error represents the direct angular difference between the robot's current orientation and the tangent of the target path. However, aligning the heading alone is insufficient if the robot is displaced laterally from the line. To resolve this, the controller utilizes a cross-track correction term inspired by the Stanley controller's kinematic approach.

$$e(t) = heading\ error + arctan(\frac{CTE}{v(t)})$$

This term effectively converts the lateral cross-track error (CTE) (the perpendicular distance from the robot's center to the closest point on the path) into a desired heading adjustment.

This error is then fed directly into the standard PID control loop, which calculates the required steering correction, ΔV, using the proportional, integral, and derivative gains:

$$\Delta V = K_p e(t) + K_i \int e(t)dt + K_d\ \dot{e}(t)$$

This differential application seamlessly adjusts the wheel velocities, shifting the robot's Instantaneous Center of Curvature (ICC) and steering it precisely back onto the reference path.

IVSI Backplane

To effectively emulate a distributed robotic framework, we utilize the Innexis Virtual System Interconnect (IVSI) backplane to link the three distinct Python components: the Simulator (Client 1), the Controller (Client 2), and the Plotter/Logger (Client 3). Rather than passing variables directly through shared memory, the system mandates that all communication traverses a simulated network. To achieve this, each Python client is equipped with a dedicated network port utilizing the python2DtEthernet gateway. This gateway is explicitly responsible for taking the standard component signals, encapsulating them into simulated Ethernet packets, and physically routing these frames across the VSI backplane to the corresponding destination components.

To establish this simulated local area network, each client port is strictly configured with its own unique MAC address and an IPv4 address residing on the 192.168.1.x subnet. The actual data exchange is driven by the TCP/IP protocol via explicitly defined port sockets.

The signal routing is designed to mimic a real-world pub-sub data flow:

- Error Broadcasting: Client 1 acts as a TCP/IP server socket on IP port 8050 to stream the plant's calculated heading_error and cross_track_error. Client 2 operates as a client socket on the same IP port to receive these parameters.
- Logging: Client 3 concurrently operates its own server socket on IP port 8080 to independently receive these error signals from Client 1 for real-time performance tracking and metric logging.
- Actuation: Once the Controller (Client 2) processes the errors, it transmits the resulting steering_correction signal back to the Simulator (Client 1) over their established Ethernet connection.
- Termination: Once the plant has reached an area within the final waypoint a signal target_reached is broadcasted to all other clients to initiate plot saving and KPI calculation followed by termination.

By defining these specific signals and explicitly routing them through the source and destination Ethernet ports, the IVSI backplane ensures that the distributed clients communicate exactly as they would over a physical TCP/IP network. This architectural setup rigorously validates the control system's viability for real-world deployment on distributed computing hardware.

Experiments And Results

Setup

All experiments were conducted under non-ideal conditions like noisy sensor data, wheel motor mechanical inertia, and constant one-sided wheel slip causing the right wheel to always be faster than the left one. All KPIs are concerning lateral error as this is the true metric indicating Trajectory vs Reference. Straight path starts from $(x = 100, y = 300)$ and ends at $(x = 700\,, y = 300)$.

Experiment 1

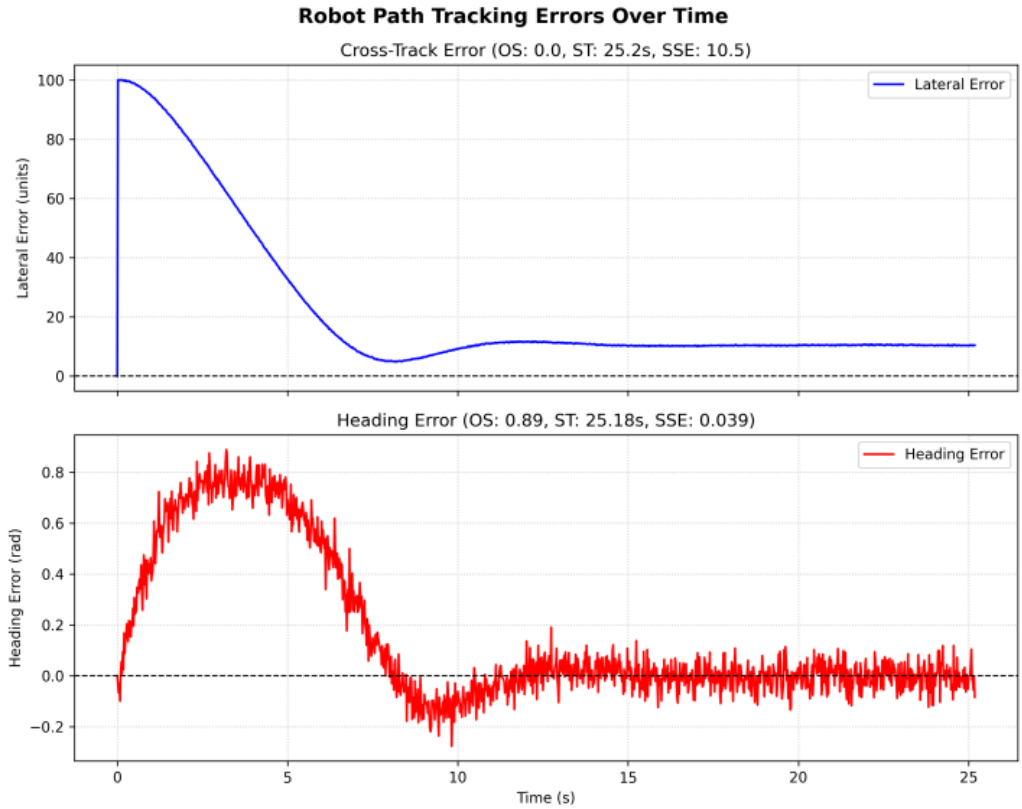
All tests in experiment 1 were following a straight reference path.

Test 1 PID Gains

$$K_p = 8.0, \qquad K_i = 0.0, \qquad K_d = 0.0$$

Test 1 - Spawn 1

$$(x, y, \theta) = (100.0, 400.0, 0)$$

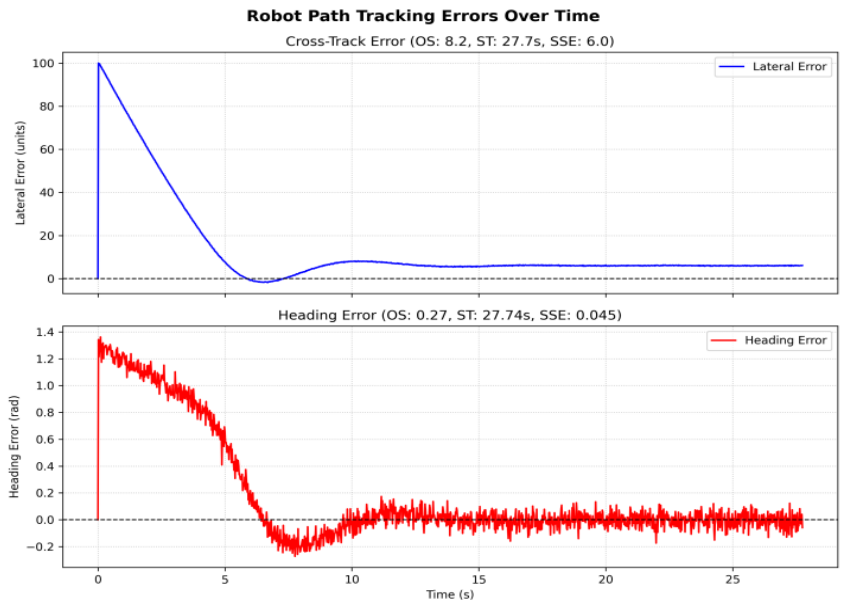


KPIs:

$$Overshoot = 0, \quad Settling\ Time = 25.2\ s, \quad SSE = 10.5\ cm$$

Test 1 - Spawn 2

$$(x, y, \theta) = (100.0, 400.0, -1.3)$$



KPIs:

$$Overshoot = approx.0\ cm, \quad Settling\ Time = 27.7\ s, \quad SSE = 6\ cm$$

Comments

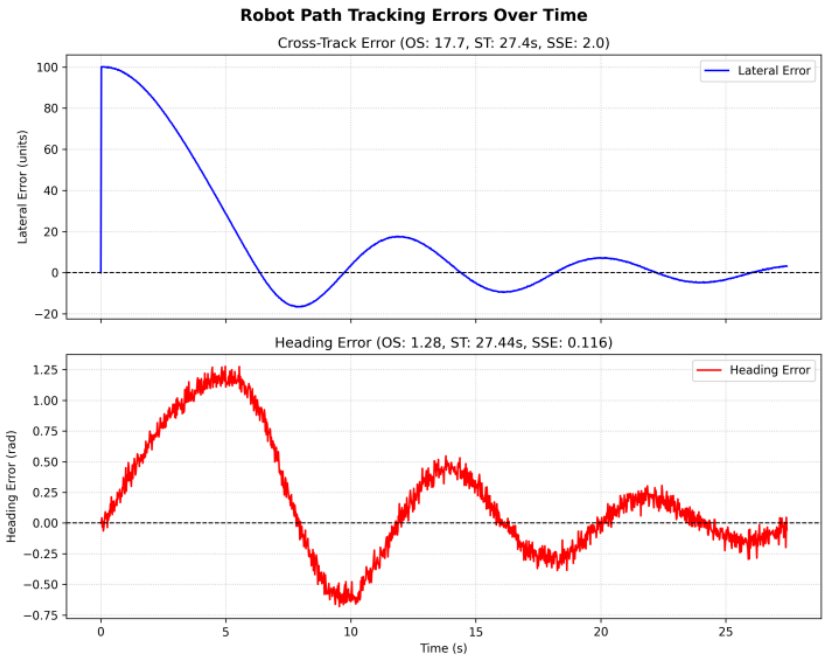
The P controller exhibits a large steady state error which is due to the simulated constant wheel slip. An Integral (I) component is essential to account for such error.

Test 2 PID Gains

$$K_p = 5.0, \qquad K_i = 1.5, \qquad K_d = 0.0$$

Test 2 - Spawn 1

$$(x, y, \theta) = (100.0, 400.0, 0)$$

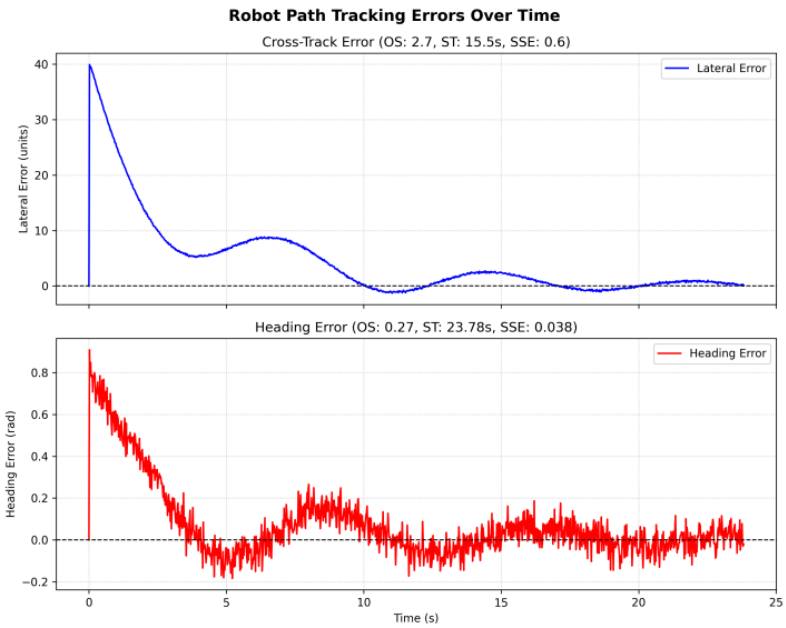


KPIs:

$$Overshoot \ = \ 17.7\ cm, \qquad Settling\ Time \ = \ 27.2\ s\ (Oscillatory), \qquad SSE \ = \ 2.0\ cm$$

Test 2 - Spawn 2

$$(x, y, \theta) = (100.0, 340.0, -0.8)$$



KPIs:

$$Overshoot \ = \ 2.7 \ cm$$
$$Settling \ Time \ = \ 15.5 \ s$$
$$SSE \ = \ 0.6 \ cm$$

Comments

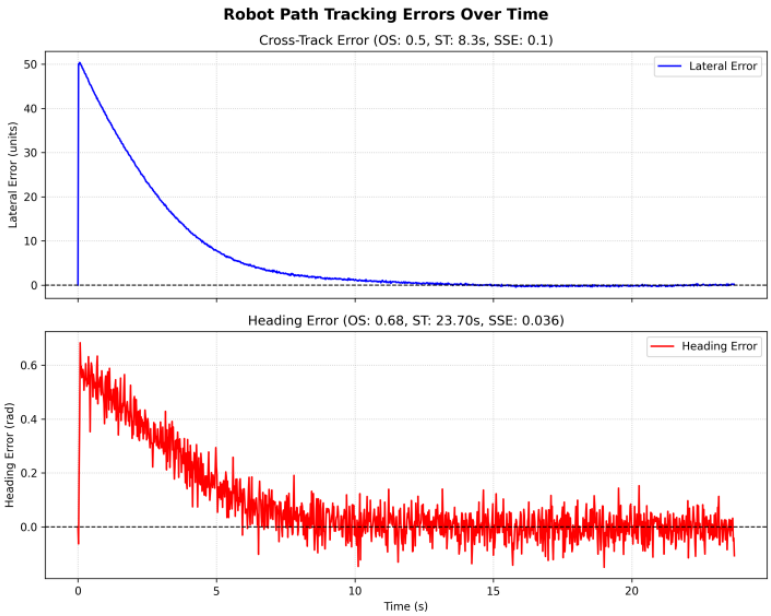
While the steady-state error is diminishing it is doing so slowly while exhibiting a damped sinusoid error response. The derivative term should help decrease these oscillations.

Test 3 PID Gains

$$K_p = 4.5, \qquad K_i = 1.47, \qquad K_d = 9.8$$

Test 3 - Spawn 1

$$(x, y, \theta) = (100.0, 350.0, -0.6)$$



KPIs:

$$Overshoot \ = \ 0.5 \ cm$$
$$Settling \ Time \ = \ 8.3 \ s$$
$$SSE \ = \ 0.1 \ cm$$

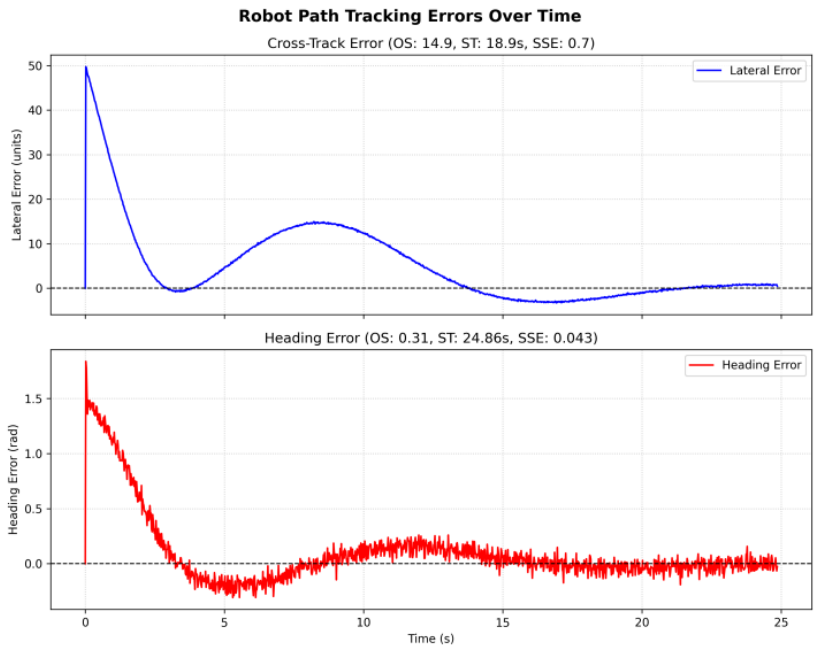
Comments

This PID gains set shows the most promising performance. It successfully eliminated the constant the wheel slip imbalance while exhibiting a critically damped error response.

Test 3 - Spawn 2

This particular test spawn has an extreme spawn position as a rigorous test for this PID gain set.

$$(x, y, \theta) = (100.0, 350.0, -0.6)$$

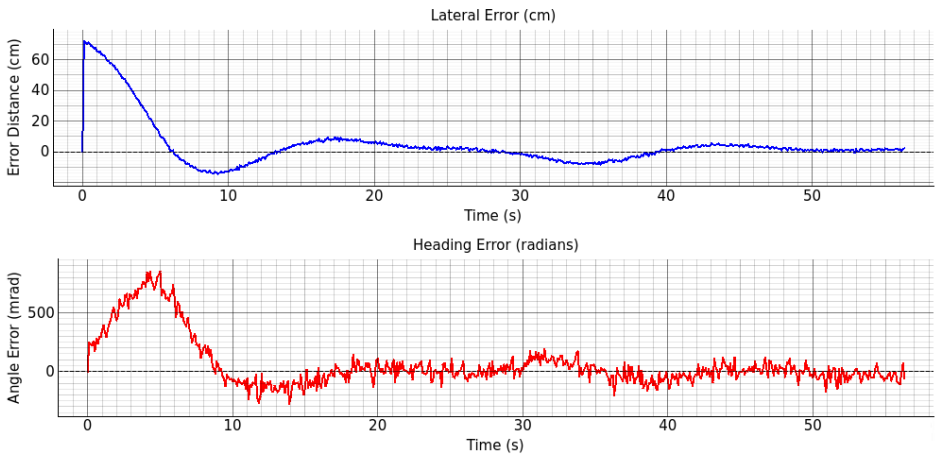


KPIs:

$$Overshoot \ = \ 14.9 \ cm$$
$$Settling \ Time \ = \ 18.9 \ s$$
$$SSE \ = \ 0.7 \ cm$$

Experiment 2

$$(x, y, \theta) = (100.0, 400.0, -0.6)$$

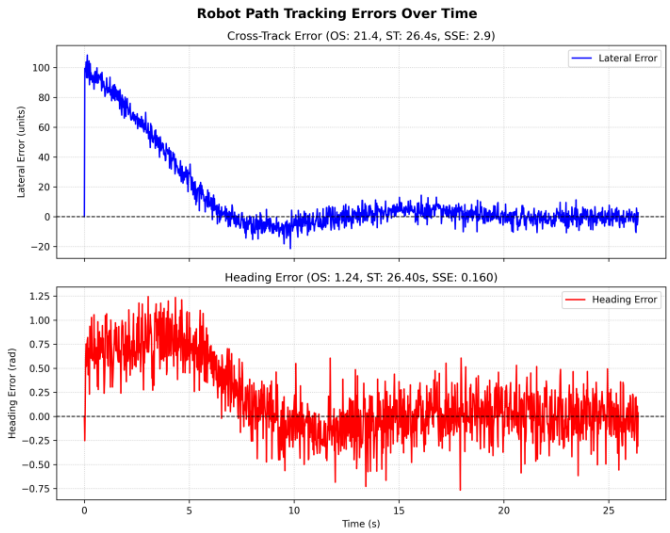


The PID Controller is able to contain the error within 11 cm of the curved path. Sudden path deviations cause minimal fluctuations (e.g. at $t = 30$) that are also contained within the 10-11 cm threshold. The constant wheel slip is quickly eliminated by the integral term and does not show up later even during turns.

Experiment 3

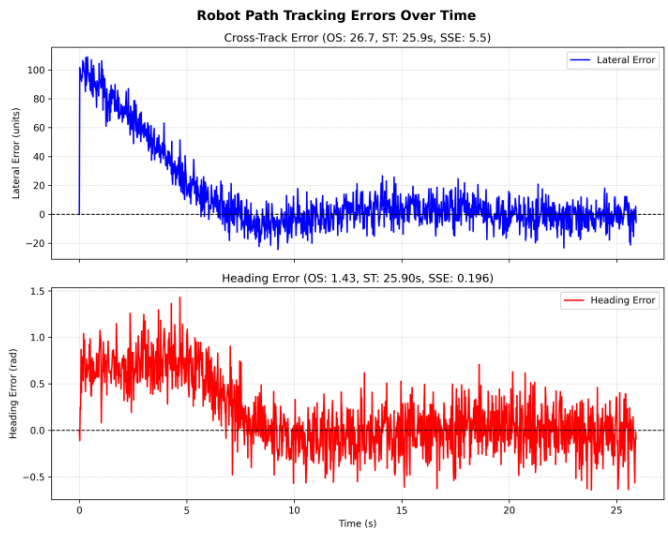
Testing the best performing PID gain set from Experiment 1 on the straight path with high noise:

$$CTE\ Noise\ std = 4, \qquad Heading\ Error\ Noise\ std = 0.2$$



The controller output shows some imperfections but the error is still contained and minimized.

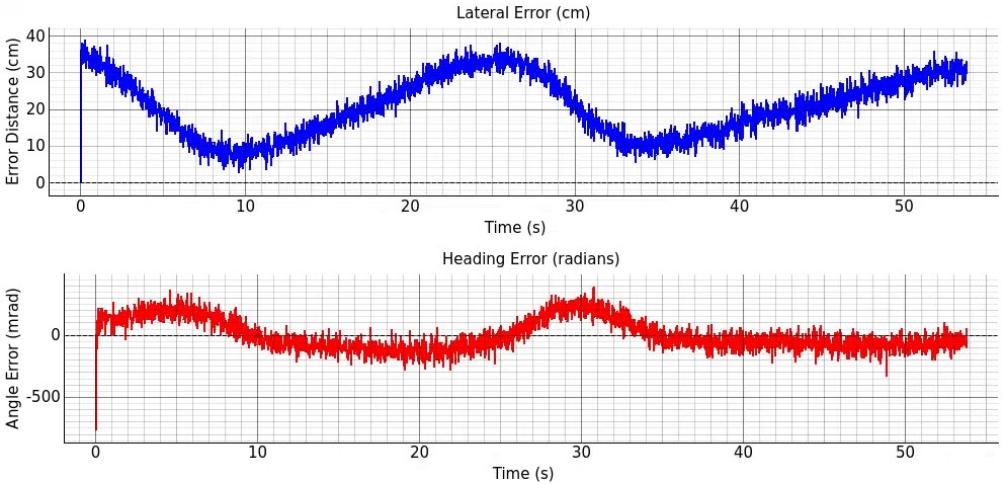
$$CTE\ Noise\ std = 8, \qquad Heading\ Error\ Noise\ std = 0.2$$



Even under extreme lateral error noise the controller is able to eliminate the constant error and to reach a minimal steady state error.

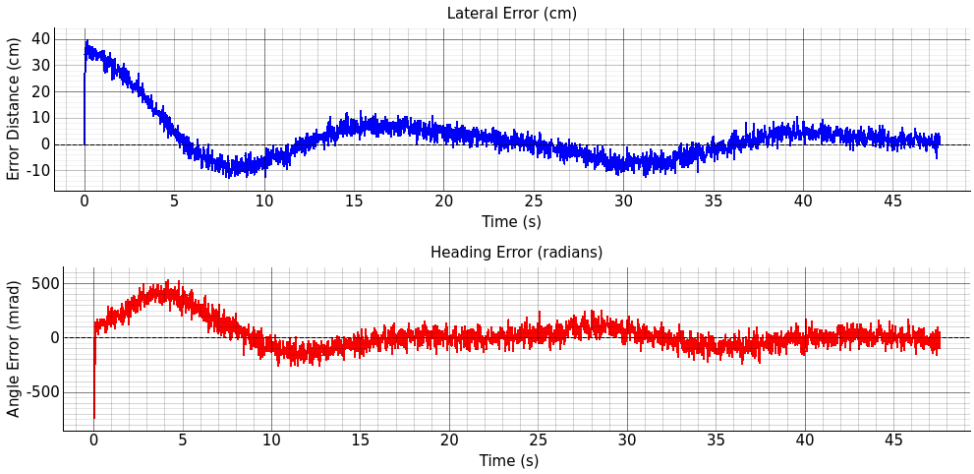
Experiment 4

PD controller



The PD controller isn't able to minimize lateral error since it is purely reactive and unable to eliminate the constant error caused by the simulated wheel slip. The result is the robot starts driving only parallel to the reference curve without being able to truly follow it.

PID controller



On the other hand, the PID controller is able to quickly eliminate the constant error through the I term and let the P and D components handle the fast reactive changes of the curved reference path.

References

CS W4733 NOTES - Differential Drive Robots. (n.d.). *Dudek and Jenkin, Computational Principles of Mobile Robotics*, 1–4. https://www.cs.columbia.edu/~allen/F17/NOTES/icckinematics.pdf

Innexis Virtual System Interconnect. (n.d.). https://www.siemens.com/en-us/products/ic/hav/innexis/virtual-system-

interconnect/#:~:text=Innexis%20virtual%20system%20interconnect%20links%20hardware%20platforms%2C,tools%20for%20the%20simulation%20of%20digital%20twin